

ACTIVIDAD 09: LAB: FILE PATH TRAVERSAL TRAVERSAL SEQUENCES STRIPPED NON-RECURSIVELY

CNO IV (T37A): Seguridad Informática

Alumno: Alejandro García Morales
Profesor: Mtro. Servando López Contreras

Contenido

1. Nombre del Laboratorio	2
2. Nivel de Dificultad	2
3. Tipo de Explotación	2
4. Objetivo del Laboratorio	2
5. Marco Teórico y Fundamentación Técnica	2
5.1. Definición Formal de la Vulnerabilidad	2
5.2. Funcionamiento en el Sistema Operativo y Servidor Web.....	2
5.3. Análisis de Impacto en la Tríada CIA (Confidencialidad, Integridad y Disponibilidad) 3	
5.4. Variantes Principales de Path Traversal	3
6. Procedimiento Paso a Paso y Uso Técnico de Burp Suite	3
Paso 1: Acceso e inicialización del entorno del laboratorio	3
Paso 2: Configuración del entorno de auditoría e inicio del navegador	4
Paso 3: Carga y mapeo visual de la aplicación objetivo	5
Paso 4: Inspección de peticiones HTTP y selección del vector en HTTP History	6
Paso 5: Aislamiento del parámetro vulnerable en el módulo Repeater	7
Paso 6: Inyección del Payload anidado y exfiltración de /etc/passwd.....	8
Paso 7: Validación y confirmación del reto resuelto.....	9
7. Registro de Evidencias Técnicas	9
8. Explicación Técnica del Payload	10
9. Mitigación Recomendada y Principios de Defensa en Profundidad	10
9.1. Abandonar el Filtrado Basado en Listas Negras	10
9.2. Implementación de Resolución de Rutas (Canonicalización).....	10
9.3. Abstracción del Sistema de Archivos (Identificadores Indirectos).....	10
10. Referencias bibliográficas.....	11

1. Nombre del Laboratorio

Lab: File path traversal, traversal sequences stripped non-recursively.

Plataforma: PortSwigger Web Security Academy.

2. Nivel de Dificultad

Practitioner.

3. Tipo de Explotación

- **Categoría principal:** Inyección de secuencias de escape de directorio (File Path Traversal).
- **Mecanismo:** Evasión de filtros de sanitización (Bypass) mediante secuencias anidadas, explotando la falta de limpieza recursiva en el servidor web.

4. Objetivo del Laboratorio

La aplicación implementa un filtro de seguridad que detecta y elimina secuencias explícitas de subida de directorio (../) presentes en el parámetro filename. El objetivo de este laboratorio es demostrar que dicha sanitización es defectuosa por no aplicarse de forma recursiva, logrando estructurar un payload anidado para evadir el filtro y extraer el archivo /etc/passwd.

5. Marco Teórico y Fundamentación Técnica

5.1. Definición Formal de la Vulnerabilidad

Catalogada como CWE-22 (Improper Limitation of a Pathname to a Restricted Directory) y ubicada en el OWASP Top 10:2021 dentro de A01:2021 (Broken Access Control), la vulnerabilidad de File Path Traversal ocurre cuando un atacante manipula referencias a archivos para acceder a directorios no autorizados. En esta variante específica, la aplicación intenta defenderse implementando una función de reemplazo, lo cual constituye una técnica de sanitización superficial conocida como "Blacklisting".

5.2. Funcionamiento en el Sistema Operativo y Servidor Web

Cuando los desarrolladores utilizan mecanismos de búsqueda y reemplazo simples para eliminar secuencias maliciosas, a menudo olvidan que el procesamiento ocurre en una única pasada (no recursiva).

Si un atacante envía la secuencia anidada://

El servidor web lee la cadena e identifica el patrón central ../.

El servidor aplica el filtro y elimina esa parte intermedia.

Las piezas restantes (.. de la izquierda y / de la derecha) se concatenan automáticamente, reconstruyendo la secuencia original ../ que luego se pasa al sistema operativo.

5.3. Análisis de Impacto en la Tríada CIA (Confidencialidad, Integridad y Disponibilidad)

- **Confidencialidad (Impacto Crítico):** Un atacante logra evadir las defensas implementadas, obteniendo acceso a archivos del sistema de archivos local (/etc/passwd).
- **Integridad (Impacto Nulo a Alto):** Al tratarse de un vector de lectura, la integridad de los ficheros se mantiene intacta.
- **Disponibilidad (Impacto Medio):** Solicitar repetidamente la lectura de archivos de sistema puede saturar los hilos del servidor web, aunque no suele ser el objetivo primario de este vector.

5.4. Variantes Principales de Path Traversal

El vector abordado en este laboratorio es el Traversal sequences stripped non-recursively. Destaca por poner a prueba la lógica de la implementación defensiva, demostrando que aplicar filtros mediante sustitución simple es insuficiente ante atacantes que comprenden el comportamiento del compilador y las rutinas de reemplazo de cadenas del backend.

6. Procedimiento Paso a Paso y Uso Técnico de Burp Suite

Paso 1: Acceso e inicialización del entorno del laboratorio

Se ingresa al portal de PortSwigger Web Security Academy en la sección del laboratorio "File path traversal, traversal sequences stripped non-recursively". Tras identificar que se trata de un nivel Practitioner, se inicializa la máquina vulnerable haciendo clic en el botón naranja "Access the lab".

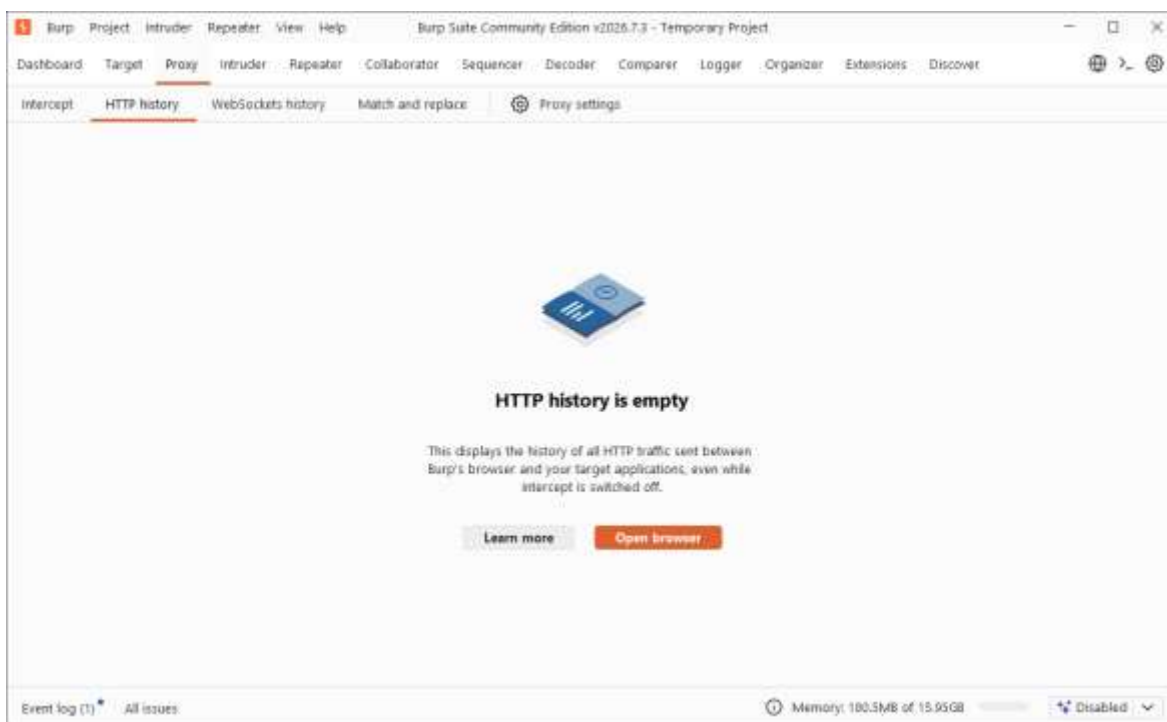
Lab: File path traversal, traversal sequences stripped non-recursively

The screenshot shows the lab interface with a blue header containing the title. Below the header, there's a navigation bar with a 'LAB' button and a 'Not solved' status. The main content area contains the lab description: 'This lab contains a path traversal vulnerability in the display of product images. The application strips path traversal sequences from the user-supplied filename before using it. To solve the lab, retrieve the contents of the /etc/passwd file.' At the bottom, there is an orange 'ACCESS THE LAB' button.

Evidencia 01: Interfaz inicial del laboratorio en la plataforma Web Security Academy, detallando la premisa del filtro no recursivo y el botón Access the lab.

Paso 2: Configuración del entorno de auditoría e inicio del navegador

Se abre Burp Suite Community Edition y se accede a la pestaña Proxy > HTTP history. Verificando que el historial se encuentre vacío ("HTTP history is empty"), se lanza el navegador integrado mediante la opción "Open browser" para capturar pasivamente todo el tráfico.



Evidencia 02: Ventana principal de Burp Suite con el historial HTTP listo para iniciar la recolección de tráfico web.

Paso 3: Carga y mapeo visual de la aplicación objetivo

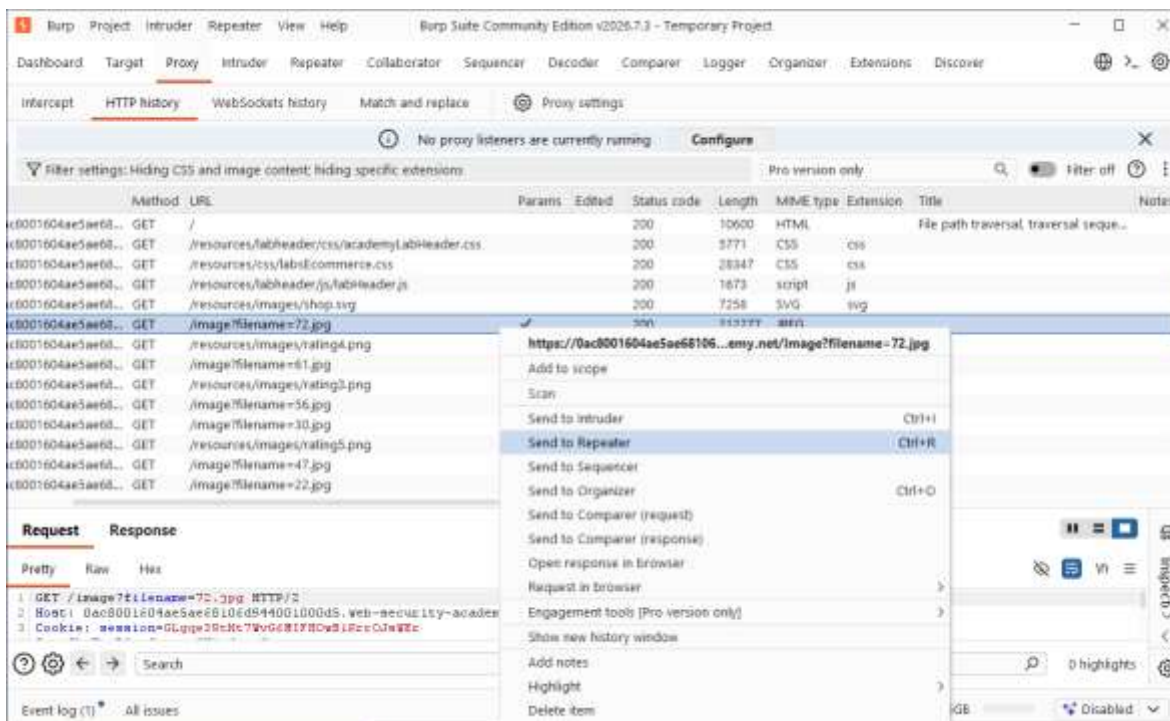
Se inspecciona visualmente la tienda en línea generada ("WE LIKE TO SHOP"). Se confirma que la aplicación expone imágenes dinámicas de los productos de forma similar a los laboratorios anteriores.



Evidencia 03: Vista general de la aplicación web comercial renderizada en el navegador integrado de Burp Suite.

Paso 4: Inspección de peticiones HTTP y selección del vector en HTTP History

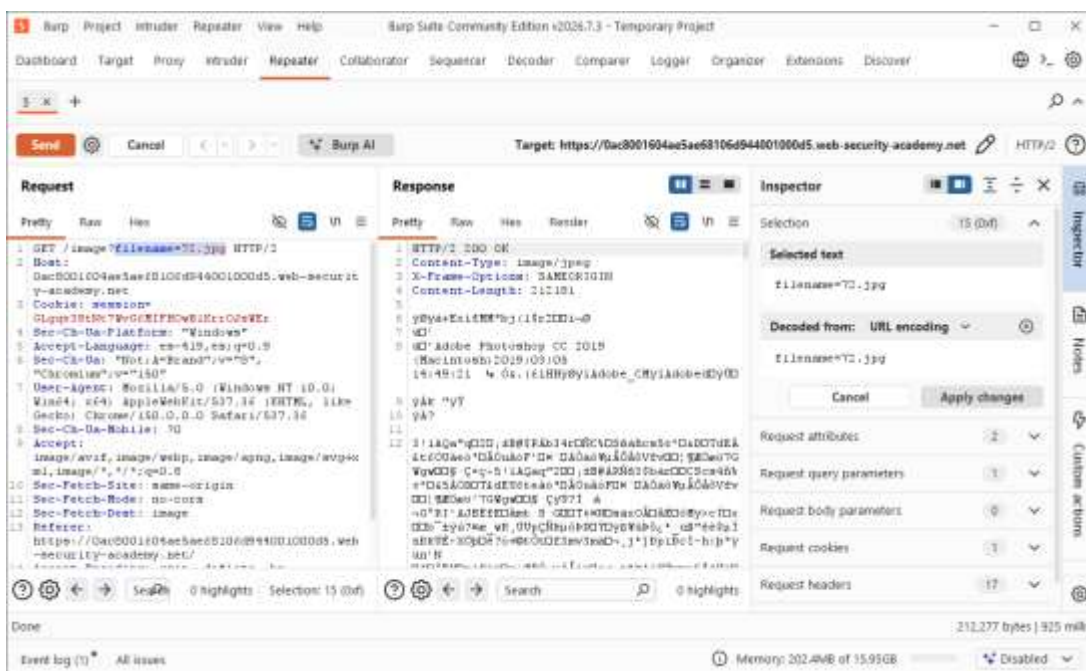
Al regresar al módulo Proxy, se examina el flujo de solicitudes. Se identifica que la aplicación carga imágenes haciendo uso del endpoint vulnerable, ejemplificado en la solicitud GET /image?filename=72.jpg. Se hace clic secundario sobre esta petición y se selecciona "Send to Repeater" (Ctrl+R).



Evidencia 04: Historial de Burp Suite mostrando la identificación de la solicitud GET vulnerable y su derivación al entorno de pruebas Repeater.

Paso 5: Aislamiento del parámetro vulnerable en el módulo Repeater

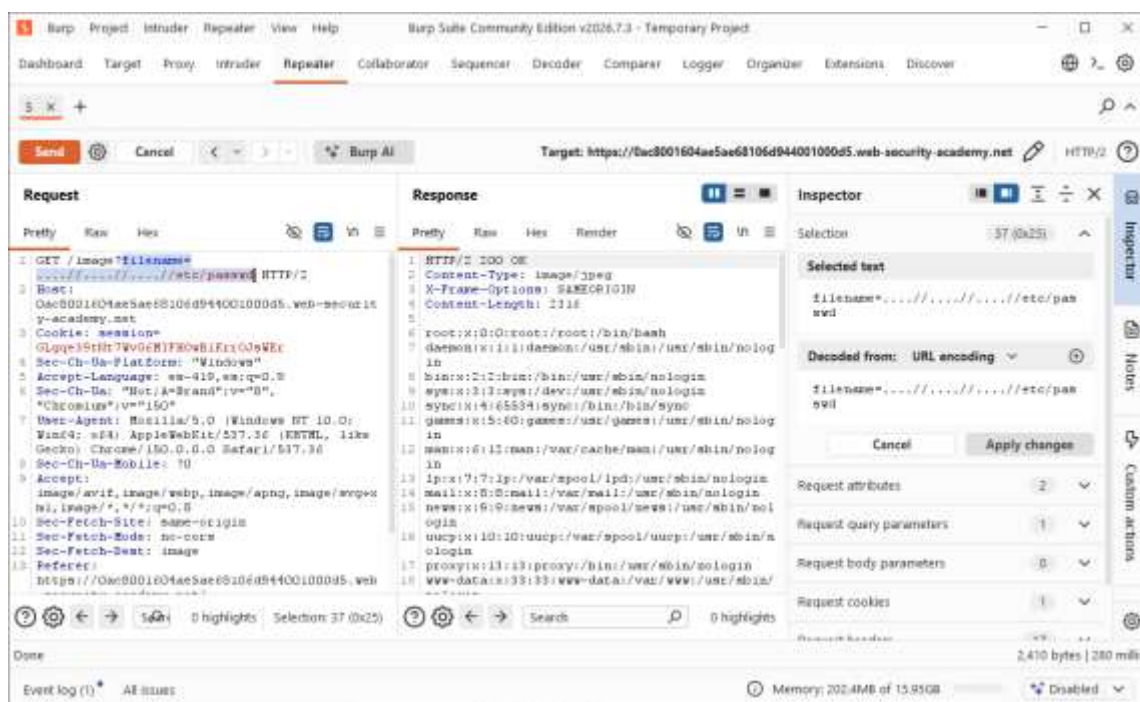
En Repeater, se aísla el parámetro filename=72.jpg en el panel Inspector. Si en este punto se intentara enviar el payload clásico (.././../etc/passwd), el servidor respondería con error o con la imagen rota, demostrando que el filtro está activo y elimina las secuencias consecutivas.



Evidencia 05: Panel de Repeater aislando la petición original, mostrando el parámetro filename intacto y la respuesta HTTP 200 OK del servidor (image/jpeg).

Paso 6: Inyección del Payload anidado y exfiltración de /etc/passwd

Para evadir la restricción no recursiva, se diseña un payload anidado. Se modifica el valor del parámetro a filename=...//...//...//etc/passwd. Al enviar la petición modificada, el filtro del servidor actúa eliminando los ../ internos, pero al hacerlo, reconstruye accidentalmente la secuencia de escape legítima. El servidor procesa la ruta final y devuelve un **HTTP/2 200 OK** con el contenido íntegro de /etc/passwd, exponiendo a usuarios críticos como root y www-data.

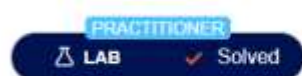


Evidencia 06: Ataque exitoso en Repeater. A la izquierda, se observa el payload anidado evadiendo el filtro. A la derecha, el servidor retorna el listado de usuarios de Linux.

Paso 7: Validación y confirmación del reto resuelto

Se recarga la ventana principal del navegador. La plataforma PortSwigger valida la exfiltración exitosa del archivo mediante la mecánica de Bypass anidado y actualiza el encabezado del sitio a "Solved" / "Lab Solved".

Lab: File path traversal, traversal sequences stripped non-recursively



This lab contains a path traversal vulnerability in the display of product images.

The application strips path traversal sequences from the user-supplied filename before using it.

To solve the lab, retrieve the contents of the `/etc/passwd` file.



Evidencia 07: Confirmación visual del estado "Lab Solved" en color verde, validando la resolución técnica del reto.

7. Registro de Evidencias Técnicas

ID Evidencia	Elemento Requerido	Descripción Técnica
EVID-01	Inicialización del entorno	Interfaz de inicio de PortSwigger con la premisa del laboratorio y el botón de acceso.
EVID-02	Configuración del Proxy	Vista de Proxy > HTTP history indicando "HTTP history is empty" y el botón Open browser.
EVID-03	Aplicación objetivo	Interfaz web de la tienda e-commerce renderizada sin bloqueos.
EVID-04	Interceptación de tráfico	Identificación de la solicitud GET hacia <code>/image?filename=72.jpg</code> enviada al módulo Repeater.
EVID-05	Localización del vector	Identificación del parámetro filename en el entorno de pruebas, devolviendo correctamente la imagen 72.jpg.
EVID-06	Inyección y Exfiltración	Petición modificada con payload anidado (<code>....//</code>) evadiendo el filtro, obteniendo código HTTP 200 y el fichero del sistema.
EVID-07	Acreditación de resolución	Banner de la plataforma ratificando "Lab Solved".

8. Explicación Técnica del Payload

Payload Utilizado:

....//....//....//etc/passwd

Análisis de la Explotación:

1. **Punto de Inyección:** Parámetro filename en la ruta /image.
2. **Mecánica de la Sanitización:** El equipo de desarrollo implementó una regla tipo Lista Negra para buscar la subcadena exacta ../ y eliminarla. Sin embargo, utilizaron una función simple de búsqueda y reemplazo que ejecuta una única barrida sobre la cadena.
3. **Comportamiento de la Evasión (Anidación):**
 - El atacante diseña la secuencia//.
 - El servidor lee la secuencia y detecta la firma maliciosa en el medio:
.(../)./
 - El servidor elimina la porción que coincide con su filtro.
 - Lo que el servidor no previó es que, al eliminar el centro, los caracteres adyacentes colapsan y se unen, formando dinámicamente un nuevo ../.
 - Como el filtro no es recursivo (no vuelve a revisar la cadena después de alterarla), esta nueva secuencia maliciosa pasa intacta al sistema de archivos.
4. **Por qué funciona el Bypass:** Multiplicando esta técnica (....// repetido tres veces), el atacante logra generar exitosamente la secuencia ../. ../. ../ directamente en la memoria del backend una vez superado el filtro. Esto permite ascender al directorio raíz y solicitar /etc/passwd.

9. Mitigación Recomendada y Principios de Defensa en Profundidad

9.1. Abandonar el Filtrado Basado en Listas Negras

Intentar "limpiar" los datos eliminando secuencias conocidas (../, ..\) es una práctica fundamentalmente insegura, ya que los atacantes continuamente encuentran nuevas formas de codificación o estructuración (como las secuencias anidadas demostradas).

9.2. Implementación de Resolución de Rutas (Canonicalización)

El servidor debe utilizar APIs del lenguaje que resuelvan la ruta absoluta real del fichero (canonicalización) antes de proceder a abrirlo. Lenguajes como Java disponen de File.getCanonicalPath() o PHP de realpath(). Una vez resuelta la ruta definitiva y procesadas todas las ambigüedades, el backend debe verificar que esta ruta resultante comience explícitamente con el directorio base autorizado (/var/www/images/).

9.3. Abstracción del Sistema de Archivos (Identificadores Indirectos)

La solución definitiva para prevenir el Path Traversal es evitar que el cliente tenga conocimiento directo de los nombres de los archivos en disco. El sistema debe asignar identificadores únicos a cada recurso almacenado en la base de datos.

Cuando se recibe una solicitud, el servidor mapea de forma segura el ID al recurso local, aislando completamente las APIs de IO del control del usuario.

10. Referencias bibliográficas

- MITRE Corporation. (2024). *CWE-22: Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')*. Common Weakness Enumeration. <https://cwe.mitre.org/data/definitions/22.html>
- Open Web Application Security Project. (2021). *A01:2021 - Broken Access Control*. OWASP Top 10. https://top10.owasp.org/2021/A01_2021-Broken_Access_Control/
- PortSwigger. (s. f.). *Lab: File path traversal, traversal sequences stripped non-recursively*. PortSwigger Web Security Academy. <https://portswigger.net/web-security/file-path-traversal/lab-sequences-stripped-non-recursively>